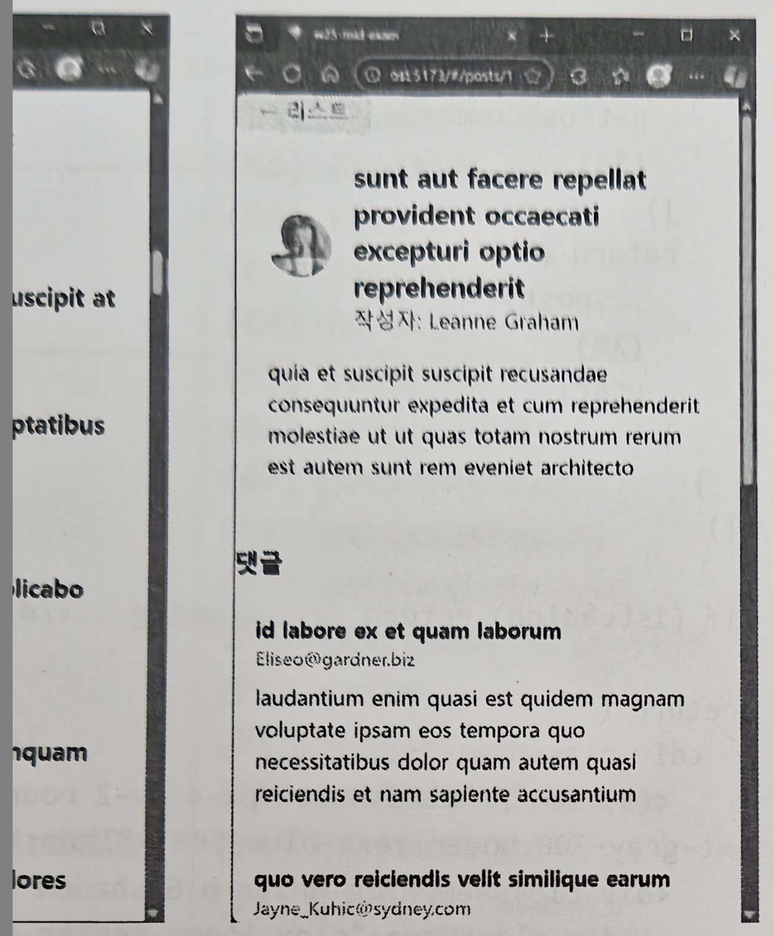
kit 금오공과대학교 시험지

	성명		학번		학년	
서버 프로그래밍		감독위원		(인)		(인)

JSONPlaceholder REST API에 접근하여 스트를 출력하고 게시글을 누르면, 상세 화면 시글 내용과 댓글(comment)들을 출력하는 프 트를 보기의 번호를 채워서 완성하시오.



App.jsx

```
import { (2), (3) } from 'react-router-dom'
import (4) from './pages/(4)'
import PostDetail from './pages/PostDetail'

function App() {
  return (
    <(2)                                                    >
      <(3)              path="/" element={<(4)              />} />
      <(3) path="/posts/(5)          " element={<PostDetail />} />
    </(2)>
  )
}

export default App
```

api/postsApi.js

```
import axios from 'axios'

const api = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
})

export const getPostsList = async () => {
  const res = await api.get('/posts')
  return res.data
}

export const getPostById = async (id) => {
  const res = await api.get(`/posts/${id}`)
  return res.data
}

export const (6)                          = async (postId) => {
  const res = await api.get(`/posts/${postId}/comments`)
  return res.data
}

export const getUsers = async () => {
  const res = await api.get('/users')
  return res.data
}

export const getUserById = async (id) => {
  const res = await api.get(`/users/${id}`)
  return res.data
}
```

vite.config.js

```
} from 'vite'
tejs/plugin-react'
m '@tailwindcss/vite'

nfig/
onfig({
ailwindcss()],
```

index.css

main.jsx

```
from 'react'
from 'react-dom/client'
ryClientProvider } from '@tanstack/react-query'
                  } from 'react-router-dom'

.jsx'

w QueryClient()

etElementById('root')).render(


ider client={queryClient}>

vider>
```

https://jsonplaceholder.typicode.com/posts

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati
excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur
expedita et cum\nreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
(생략_)
```

https://jsonplaceholder.typicode.com/posts/1

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati
excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur
expedita et cum\nreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
```

★ 부정행위를 한 자는 학생상벌규정에 의하여 징계 처분한다 ★

pages/PostList.jsx

```jsx
import { (7)                                    } from '@tanstack/react-query'
import { getPostsList, getUsers } from '../api/postsApi'
import { (8)                                    } from 'react-router-dom'

export default function PostList() {
  const { data: postList, (9)                                    } = (7)({
    queryKey: ['posts'],
    queryFn: (10)                                                  () => {
      const [posts, users] = await (11)                                          ([
        getPostsList(),
        getUsers(),
      ])

      // user id를 key로 빠르게 접근하기 위한 map 생성
      // Before:
      // const users = [
      //   { id: 1, name: 'user1' },
      //   { id: 2, name: 'user2' }
      // ]

      // users.map(user => [user.id, user])에 의해:
      // [
      //   [1, { id: 1, name: 'user1' }],
      //   [2, { id: 2, name: 'user2' }]
      // ]

      // Object.fromEntries()에 의해:
      // {
      //   1: { id: 1, name: 'user1' },
      //   2: { id: 2, name: 'user2' }
      // }
      const (12)                                  = Object.fromEntries(
        users.map(user => [user.id, user])
      )

      // 각 post에 user 정보 추가
      // Before:
      // const posts = [
      //   { id: 1, userId: 1, title: 'post1' },
      //   { id: 2, userId: 2, title: 'post2' }
      // ]
      // After:
      // const posts = [
      //   { id: 1, userId: 1, title: 'post1', user: { id: 1, name: 'user1' } },
      //   { id: 2, userId: 2, title: 'post2', user: { id: 2, name: 'user2' } }
      // ]
      return posts.map(post => ({
        ...post,
        user: usersMap[post.userId],
      }))
    },
  })

  if (isLoading) return <div>Loading...</div>

  return (
    <div className="space-y-4">
      <h1 className="text-2xl font-bold mb-4">게시글</h1>
      {postList.(13)                                    (post => (
        <(8)
          to={`/posts/${post.id}`}
          key={post.id}
          className="block p-4 bg-white shadow rounded hover:bg-gray-100 transition"
        >
          <div className="flex items-center gap-4">
            <(14)                                                    
              src={`https://i.pravatar.cc/40?u=${post.user.id}`}
              alt={post.user.name}
              className="rounded-full"
              width={40}
              height={40}
            />
            <div>
              <h2 className="text-lg font-semibold">{post.title}</h2>
              <p className="text-gray-500 text-sm">작성자: {post.user.name}</p>
            </div>
          </div>
        </(8)>
      ))}
    </div>
  )
}
```

pages/PostDetail.jsx

```jsx
import { (7) } from '@tanstack/react-query'
import { (15)                                    , (8) } from 'react-router-dom'
import {
  (17),
  getPostComments,
  (18),
} from '../api/postsApi'

export default function PostDetail() {
  const { id } = (15)()
  const { (16)                                                    , (9) } = (7)({
    queryKey: ['post', id],
    queryFn: (10) () => {
      const post = await (17)                                                    (id)
      const [comments, user] = await (11)([
        getPostComments(id),
        (18)                                                    (id),
      ])
      return {
        ...post,
        (19)                                                    ,
        user,
      }
    }
  })

  if (isLoading) return <div>Loading...</div>

  return (
    <div className="space-y-6">
      <(8) to="/" className="px-4 py-2 rounded-md bg-gray-300
text-gray-700 hover:text-blue-600">&larr; 리스트</(8)>
      <div className="bg-white p-6 shadow rounded">
        <div className="flex items-center gap-4 mb-4">
          <img
            src={`https://i.pravatar.cc/60?u=${data.user.id}`}
            alt={data.user.name}
            className="rounded-full"
            width={60}
            height={60}
          />
          <div>
            <h2 className="text-xl font-bold">{data.title}</h2>
            <p className="text-gray-600">작성자:
{data.user.name}</p>
          </div>
        </div>
        <p className="text-gray-800">{data.body}</p>
      </div>

      <div>
        <h3 className="text-lg font-semibold mb-2">댓글</h3>
        <div className="space-y-2">
          {data.comments.(13)((20)                                    => (
            <div key={comment.id} className="bg-gray-50 p-4
rounded shadow-sm">
              <p className="font-semibold">{comment.name}</p>
              <p className="text-sm text-gray-600">{comment.email}</p>
              <p className="text-gray-800 mt-2">{comment.body}</p>
            </div>
          ))}
        </div>
      </div>
    </div>
  )
}
```

※ 수고하셨습니다. Good Luck!

<답안 작성 요령>

- 답안으로는 번호만 쓸 것 (주관식으로 쓰지 말 것. 번호 이외의 답을 적으면 감점 가능)
- 틀린 경우 감점되므로, 모를 경우에는 답안을 비워둘 것 (쓰지 않으면 감점 없음, 오답을 적으면 1점 감점)
- 유사한 답이 있는 경우에는 구체적이고 정확한 답을 골라 쓸 것 (만약, 중복된 보기가 있으면 한 개만 쓸 것)
- 답이 보기에 없으면 (스펠링이나 대소문자 틀린 경우 포함) "없음"을 쓰고 주관식으로 적을 것 (가능성 매우 낮음)

<보기 (오름차순 정렬됨) >

(1) $detail
(2) $postDetail
(3) ...id
(4) :id
(5) [comments, user]
(6) [id]
(7) {id}
(8) a
(9) abstract class
(10) action
(11) adapter class
(12) allItems
(13) allItems.findAll()
(14) allItems.list
(15) allItems.value
(16) as
(17) async
(18) AttributeData
(19) attributeItemList
(20) attributeItems
(21) attributeItems.findAll()
(22) attributeList
(23) attributeList.findAll()
(24) attributeListItems
(25) Attributes
(26) Avartar
(27) avatar
(28) AvatarPhoto
(29) base
(30) baseClass
(31) bind
(32) binding
(33) cache
(34) class
(35) comment
(36) commentList
(37) comments
(38) connect
(39) const function
(40) const res =
(41) ConstraintLayout.Button
(42) constructor
(43) container
(44) currentId
(45) data
(46) data: [comments, user]
(47) data: commentList
(48) data: comments
(49) data: post
(50) data: postDetail
(51) data: postDetailInfo
(52) Data<Item>
(53) Data<List<Item>>
(54) dataBinding
(55) detail
(56) display
(57) entries
(58) enum
(59) error
(60) expand
(61) findAll
(62) findId
(63) forEach
(64) foreach
(65) function
(66) getAllItems
(67) getArrayResult
(68) getAxiosResult
(69) getCommentsById
(70) getCommentsByPostId
(71) getId
(72) getPost
(73) getPostByComments
(74) getPostById
(75) getPostComments
(76) getPostDetail
(77) getPostDetailById
(78) getPostDetailId
(79) getPostId
(80) getPostList
(81) getPostListWithCache
(82) getPostsAndUsers
(83) getPostsByPostId
(84) getQueryWithCache
(85) getReactRouterId
(86) getResponse
(87) getRouterDomId
(88) getRouterId
(89) getter
(90) getUser
(91) getUserById
(92) getUsers
(93) hashMap
(94) HashRouter
(95) HashSet
(96) href
(97) Html<List<Item>>
(98) HtmlTemplate
(99) hyperLink
(100) id
(101) img
(102) init
(103) init(Item)
(104) init(items)
(105) init(List<Item>)
(106) initData(List<Item>)
(107) initialize
(108) Int
(109) int
(110) interface
(111) into
(112) isLoading
(113) it
(114) Item
(115) item
(116) itemData
(117) itemData.value
(118) itemId
(119) itemList
(120) itemList.data
(121) itemList.findAll()
(122) itemList.list
(123) itemList.value
(124) itemListData
(125) items
(126) items.count
(127) items.data
(128) items.findAll()
(129) items.list
(130) items.size
(131) items.value
(132) keyMap
(133) lambda
(134) lambda function
(135) Link
(136) List
(137) List<Item>
(138) listOf
(139) make
(140) map
(141) Model
(142) new function
(143) new HashSet
(144) new Item
(145) new List
(146) new record
(147) null
(148) object
(149) objectFromEntries
(150) open
(151) override
(152) Page
(153) PageRoot
(154) Pages
(155) post.getId
(156) post.parseJson
(157) post.result
(158) postComments
(159) postDetail
(160) postDetailInfo
(161) postId
(162) PostList
(163) PostRoot
(164) posts.byId
(165) Promise.all
(166) query
(167) ReactPath
(168) ReactQuery
(169) reactQuery
(170) ReactRouter
(171) ReactRouterDom
(172) record
(173) record Item
(174) response
(175) responseArray
(176) resposeFunction
(177) result
(178) return
(179) Root
(180) Route
(181) Router
(182) RouterDom
(183) RouterDoms
(184) RouterElement
(185) RouterElements
(186) RouterInfo
(187) RouterPage
(188) RouterPages
(189) RouterPath
(190) RouterPaths
(191) Routes
(192) search
(193) static
(194) struct
(195) super
(196) TanStackQuery
(197) tanstackQuery
(198) Template<List<Item>>
(199) this
(200) tr
(201) tuple
(202) Unit
(203) useParams
(204) useQuery
(205) UserAvatar
(206) UserPhoto
(207) usersMap
(208) val
(209) var
(210) view
(211) virtual
(212) when